MODULE 2 · STATISTICS & PROBABILITY · TOPIC 2.19

A/B Test Design & Analysis

Day 30 — the last statistics session, and the only one where you are allowed to say "caused"



Boston University

BEFORE

Decide the size. The sum takes one line.

DURING

Do not look. Three bugs to watch for.

AFTER

Read the interval, not the verdict.

Prepared & presented by **Srinivasa Sai Chava**

WHY THIS SESSION

Yesterday: five explanations. Today: how to rule out four of them.

What we were stuck with yesterday

Chance · confounding · reverse causation · selection · cause.

With observational data you can argue about which one it is, but you can never settle it — because you did not control who got what.

What a coin flip buys you

If a coin decides who sees which version, the two groups differ in exactly one way on average — the thing you changed.

Confounding, reverse causation and selection are all switched off by the design. Only chance is left, and chance is what the maths handles.

Four things you will see today

1. The same users give a +21 effect observationally and +2.4 when randomised
2. A small test that “wins” overstates the lift by 3.3× — and it is still “significant”
3. Checking the dashboard 20 times turns a 5% error rate into 25%
4. On revenue, a real +10% lift is invisible at n = 1,000 — 9% power

WARM-UP · 5 MINUTES

Four questions from Day 29

1	**What does r actually measure?**	→ Tightness around a STRAIGHT line. Nothing else.
2	**r = 0.3 — how much is explained?**	→ 9%. Report r^2, always.
3	**Name the five explanations.**	→ Chance, confounding, reverse, selection, cause.
4	**Which one does a coin flip remove?**	→ Three of them — and that is today.

Everything from Day 23 onwards gets used today at once: a mean, a standard error, a sampling distribution, a p-value, a confidence interval, power, and the multiplicity problem. An A/B test is the whole module applied to one decision.

SESSION PLAN

How today's two hours are structured

1. **What an A/B test is** — 15 min
   And what randomising actually buys

2. **Size it before you run it** — 20 min
   MDE, power, and the winner's curse

3. **Running it without breaking it** — 20 min
   Peeking, SRM, novelty

4. **Analysing it** — 20 min
   The test, revenue, and segments

5. **Deciding and reporting** — 15 min
   Ship, kill, or learn nothing

6. **Practice notebook** — 30 min
   You design one, then break one

Section 2 is the one that saves you the most time in practice. Most failed A/B tests were doomed before a single user saw them, because nobody did the sum.

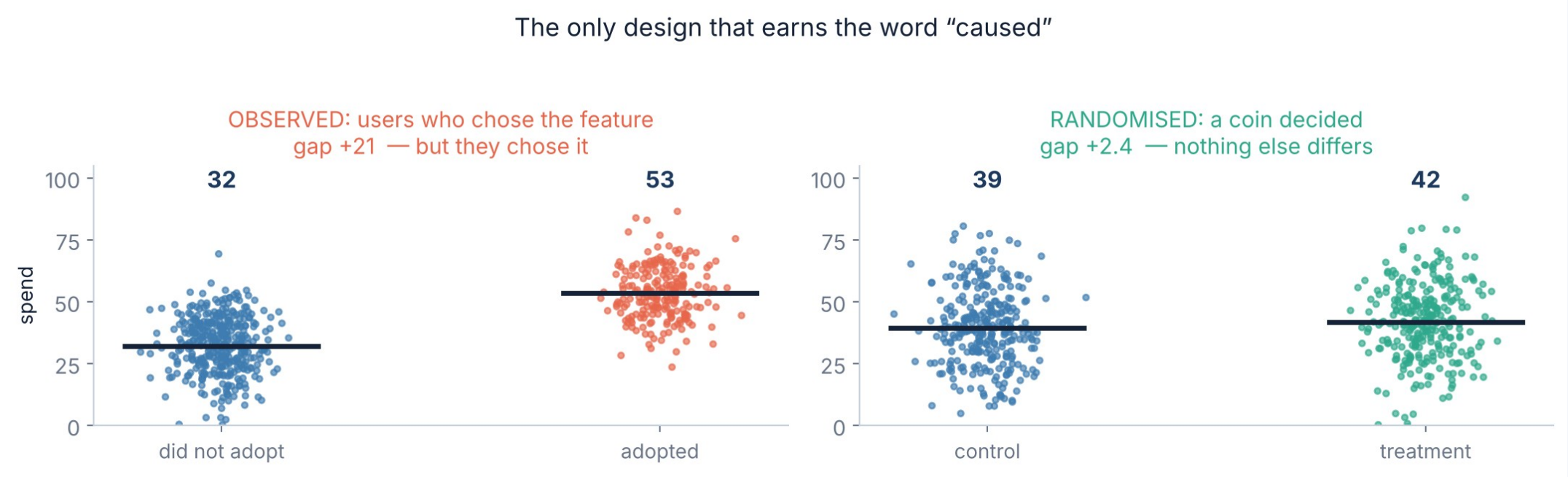
1

What an A/B Test Is

The only design that earns the word “caused” · 15 minutes

- Randomisation
- The three decisions
- The unit

⭐ A coin flip is the entire trick



Left: the observational answer

Users who adopted the feature spend 21 more. But engaged users are the ones who adopt — the gap mostly measures who they already were.

Right: the randomised answer

Same users, same engagement. A coin assigns the arm, so engagement is balanced on average. The true effect built into this simulation was +2.0.

What the coin fixes — and what it does not

✓ Switched off by the design

- Confounding — every variable, including the ones you never measured or thought of, is balanced on average
- Reverse causation — the coin came first, in time, so the outcome cannot have caused the assignment
- Selection — you decide who is in the test, before anyone has an outcome

× Still entirely your problem

- Chance — the coin balances things on average, not in your particular test
- Too small a test — randomising badly-sized arms answers nothing, correctly
- Measuring the wrong thing — a clean test of a metric nobody cares about
- Short horizon — you measured a week, and shipped for a year

"Balanced on average" is the exact claim. In any one test the groups will differ by chance — that is precisely the uncertainty the p-value and the interval are measuring.

Three decisions, all made before the test starts

1

The unit

What gets randomised — almost always the USER, not the session or the pageview

Randomise by pageview and the same person sees both versions. They notice, and their pageviews are not independent, so every p-value is wrong.

2

The metric

ONE primary metric, written down before you look

Everything else is a secondary metric or a guardrail, and secondary metrics cannot win the test. They can only lose it.

3

The size

How many users, worked out from the smallest effect worth shipping

This is section 2. It takes one line of code and it is the step that is almost always skipped.

Write all three in a one-page plan and have somebody sign it. After the data arrives, every one of these decisions becomes negotiable — and that is how tests get p-hacked without anyone deciding to cheat.

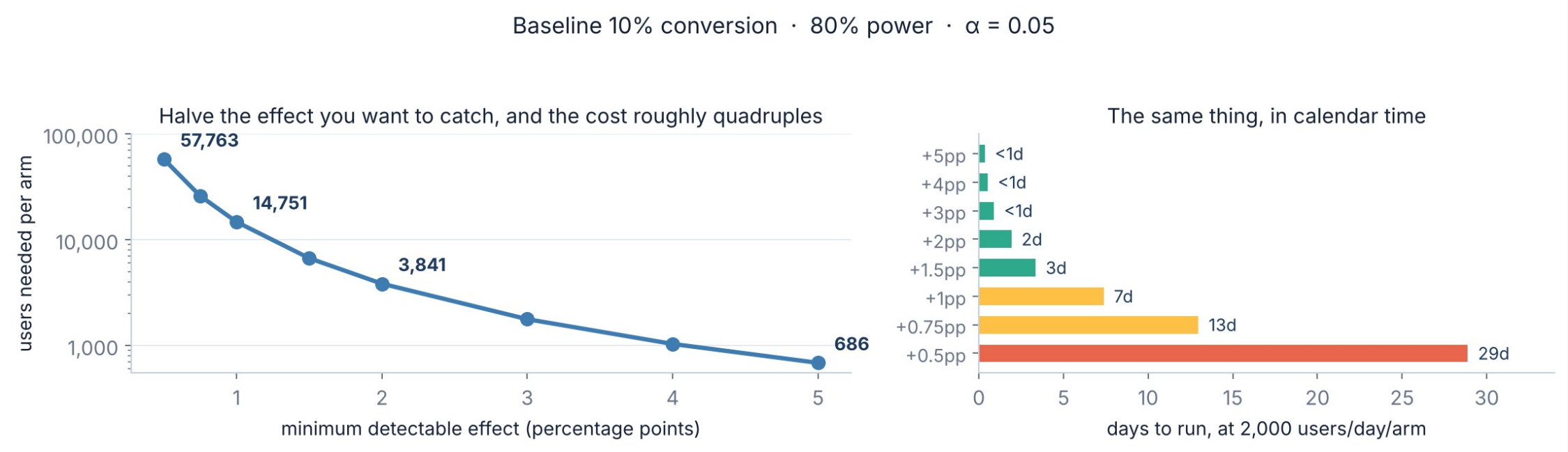
2

Size It Before You Run It

The sum that saves the whole test · 20 minutes

- MDE and sample size
- The formula
- The winner's curse

Not “how many users?” but “how small an effect?”



Start from the business, not the stats

Ask: what is the smallest lift that would make us ship this? That is the MDE. Everything else follows from it mechanically.

The quadrupling rule

Halve the effect you want to catch and you need roughly four times the users. +2pp costs 3,841 per arm; +1pp costs 14,751; +0.5pp costs 57,763 and a month.

The sum itself — one line, four inputs

```
from scipy import stats
import numpy as np

def n_per_arm(p0, mde, power=0.80, alpha=0.05):
    """Users needed in EACH arm of a two-sided proportion test."""
    p1   = p0 + mde
    pbar = (p0 + p1) / 2
    za   = stats.norm.ppf(1 - alpha/2)     # 1.96
    zb   = stats.norm.ppf(power)           # 0.84
    num  = (za*np.sqrt(2*pbar*(1-pbar)) +
            zb*np.sqrt(p0*(1-p0) + p1*(1-p1)))**2
    return int(np.ceil(num / mde**2))

n_per_arm(0.10, 0.01)      # -> 14751
```

The four inputs

`p0`

your current rate — look it up, do not guess

`mde`

smallest lift worth shipping — a business call

`power`

0.80 — chance of catching a real effect

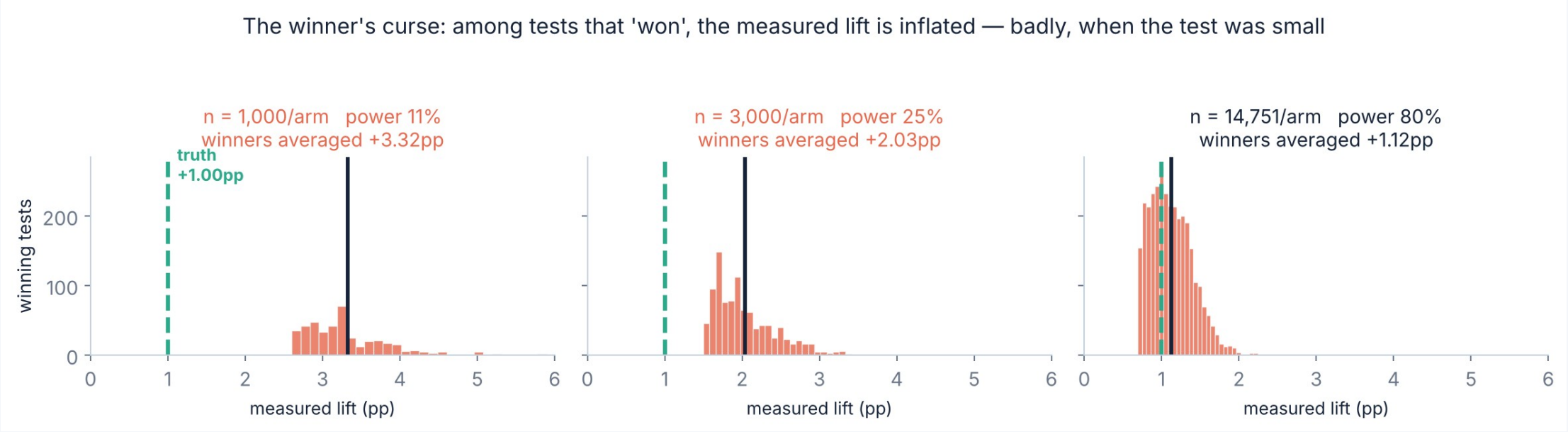
`alpha`

0.05 — chance of a false alarm

Checked against statsmodels

This hand-written version gives 14,751 where statsmodels' NormalIndPower gives 14,745 — a 0.04% difference from a different variance convention. Either is fine. Write it yourself once so you know what the inputs mean, then use the library.

⚠️ An underpowered test does not just miss — it lies about the size



Every one of those tests was correct

The true lift is +1.00pp in all three panels. At n = 1,000 the only tests that reach p < 0.05 are the ones where noise happened to push the gap up — so the winners average +3.32pp. That is a 3.3× overstatement, with a significant p-value attached.

Why this matters on Monday

This is why shipped features so often fail to deliver the lift the test promised — and why a properly sized test (right-hand panel, 80% power) overstates by only 1.12×. Power is not just about finding effects. It is about measuring them.

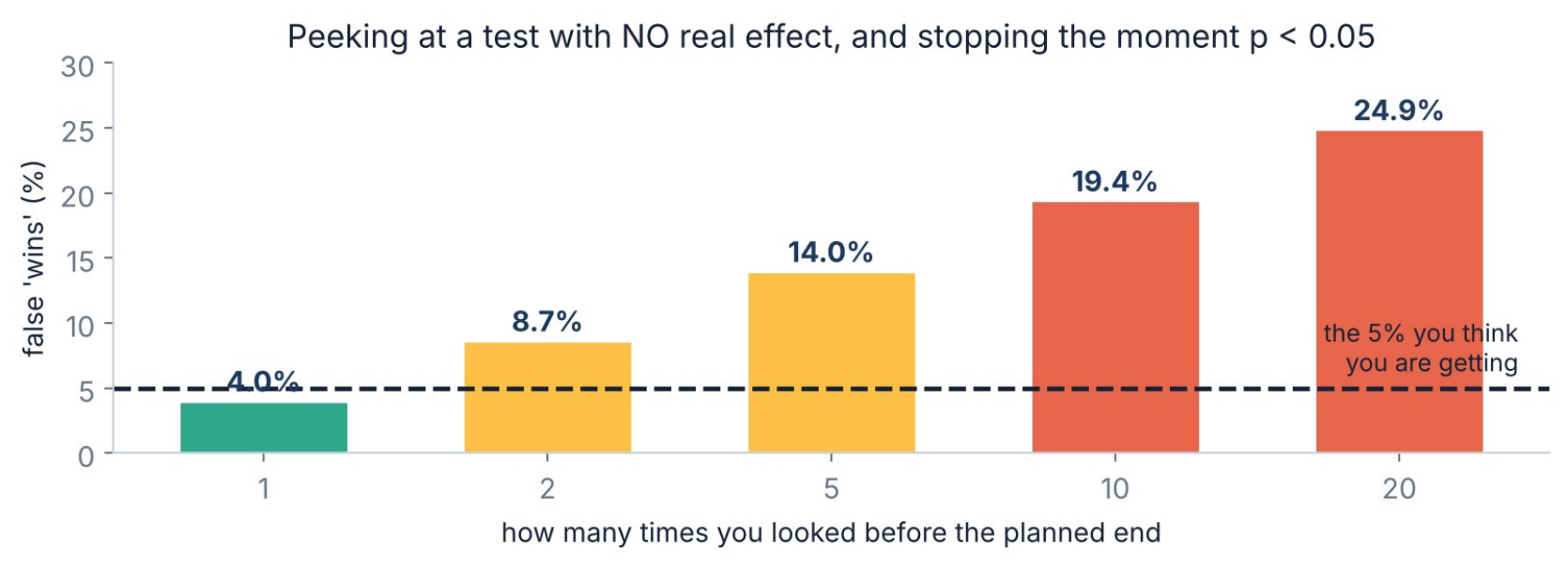
3

Running It Without Breaking It

Three ways a clean design goes wrong · 20 minutes

- Peeking
- Sample ratio mismatch
- Novelty

⚠️ Watching the dashboard is a statistical procedure



Why it happens

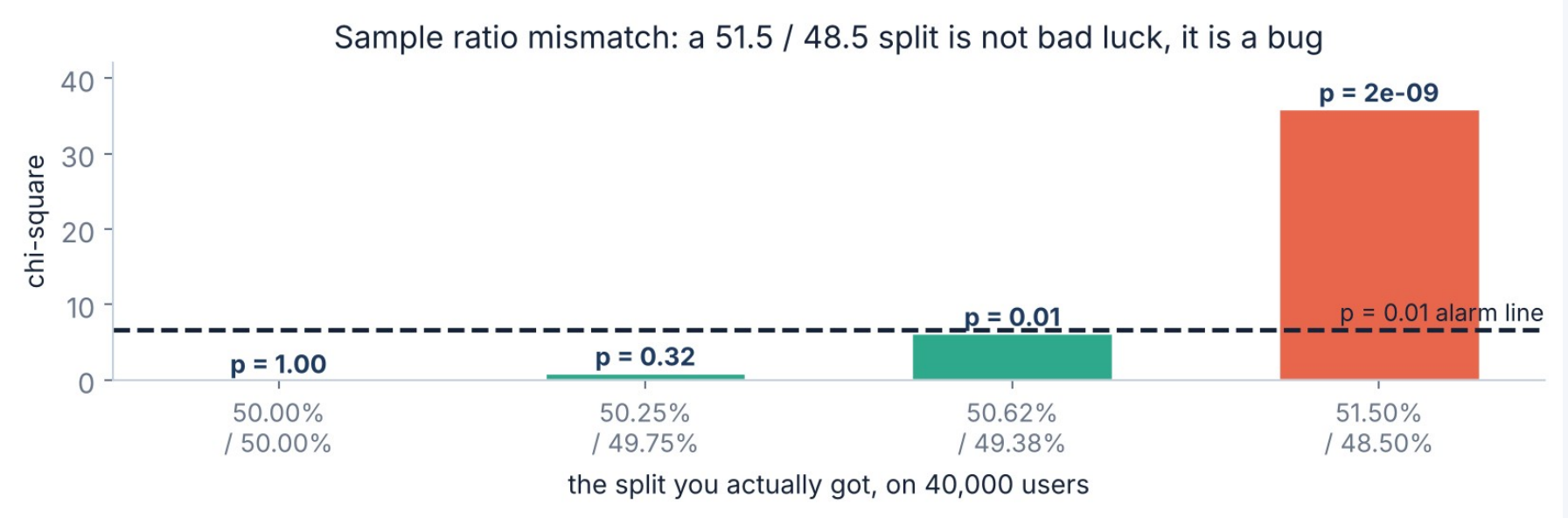
The p-value wanders as data arrives. Stop the moment it dips under 0.05 and you have given yourself twenty chances to catch a fluctuation instead of one. Nobody intends this — it is just what happens when a dashboard exists.

The three honest options

1. Fix the end date from the sample-size sum and do not look.
2. Look, but only at guardrails — crashes, errors, refunds.
3. Use a method built for it: sequential testing, or always-valid p-values. Not a plain t-test checked daily.

3.2 · SRM

Sample ratio mismatch — the best five minutes you will spend



What it is

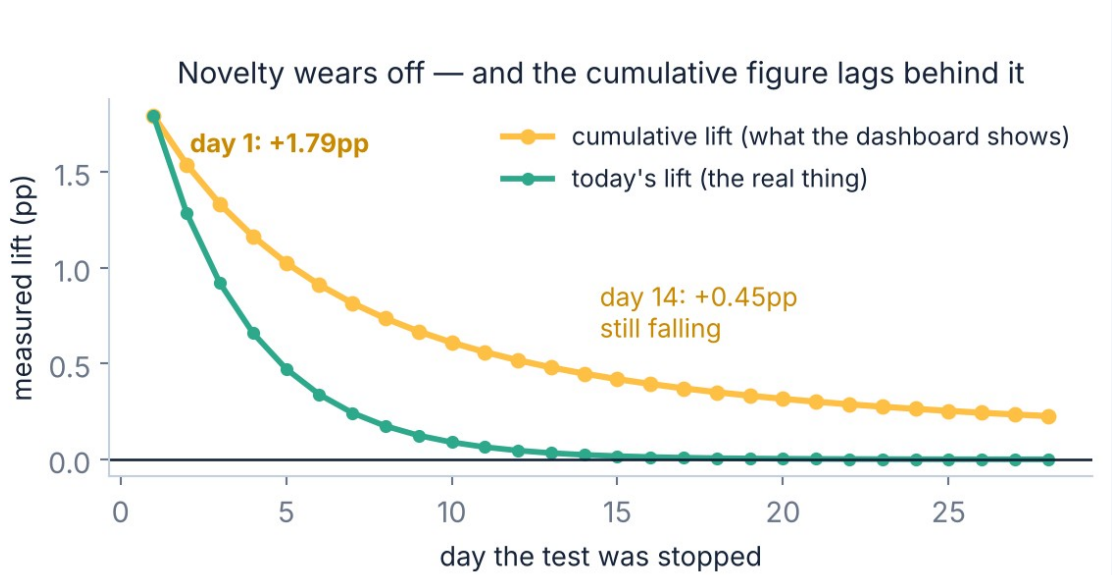
You asked for a 50/50 split. Check whether you got one, with a chi-square test on the two counts. On 40,000 users, 51.5/48.5 gives p = 2e-9 — that cannot be bad luck.

⚠️ It invalidates the whole test

A bad split means users were lost unevenly — a redirect that timed out, a bot filter that hit one arm, logging that dropped slow sessions. Whatever was lost was not random, so the groups are no longer comparable. Fix the bug and rerun; do not analyse it.

⚠️ Novelty, weekends, and why you run whole weeks





The dashboard lags the truth

Day 1 shows +1.79pp. By day 14 the daily lift is +0.02pp — effectively gone — but the cumulative figure still reads +0.45pp, because it is still averaging in week one.

Two rules that follow

Run in WHOLE WEEKS — the baseline swings 2.1pp between weekdays and weekends, so a Monday-to-Friday test measures a different population.

And plot the DAILY lift, not only the cumulative one. A decaying daily line means you have not reached the long-run effect yet.

4

Analysing It

The easy part, and its three traps · 20 minutes

- The test
- Revenue metrics
- Segments

The analysis is five lines. That was always the easy part.

```
def two_prop(c_a, n_a, c_b, n_b, alpha=0.05):
    p_a, p_b = c_a/n_a, c_b/n_b
    diff = p_b - p_a                      # ABSOLUTE lift

    # pooled SE for the test (assumes H0 true)
    pbar = (c_a + c_b) / (n_a + n_b)
    z = diff / np.sqrt(pbar*(1-pbar)*(1/n_a + 1/n_b))
    p = 2 * (1 - stats.norm.cdf(abs(z)))

    # UNpooled SE for the interval (does not assume H0)
    se = np.sqrt(p_a*(1-p_a)/n_a + p_b*(1-p_b)/n_b)
    c  = stats.norm.ppf(1 - alpha/2)
    return diff, p, (diff - c*se, diff + c*se)
```

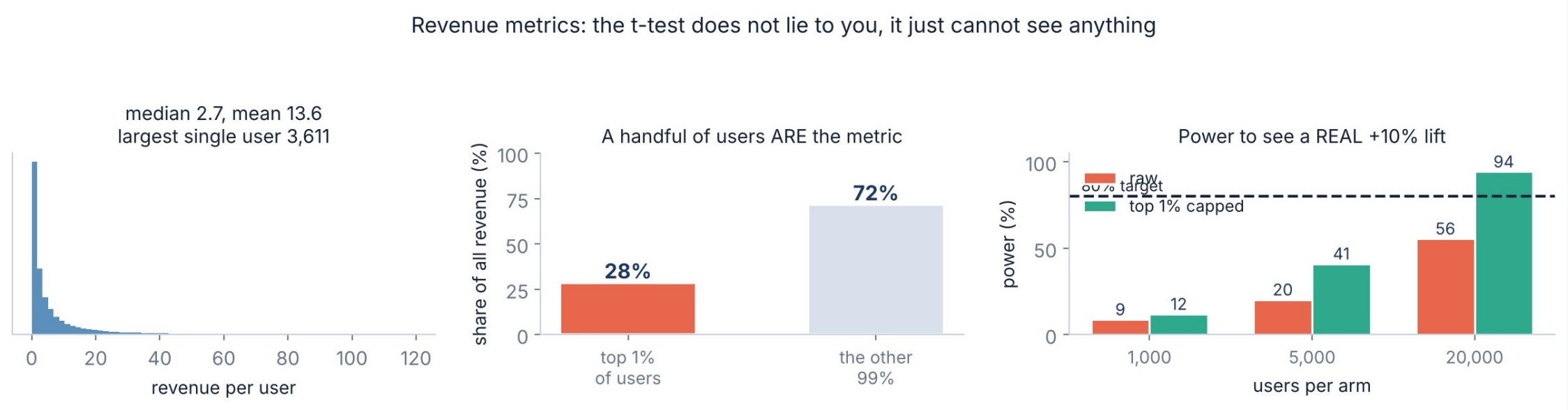
Two details worth the minute

Pooled vs unpooled SE — the test assumes the null is true, so it pools the two rates. The interval does not assume that, so it must not pool. Using one for both is a common and quiet error.

Absolute vs relative — “+1.5 percentage points” and “+15%” are the same result. Pick one, label it, and never mix them in a sentence.

For a continuous metric — time on page, items per order — use stats.ttest_ind(a, b, equal_var=False) instead. Day 28's lesson still applies: Welch, always.

⚠️ Revenue metrics: the test is honest, it is just blind



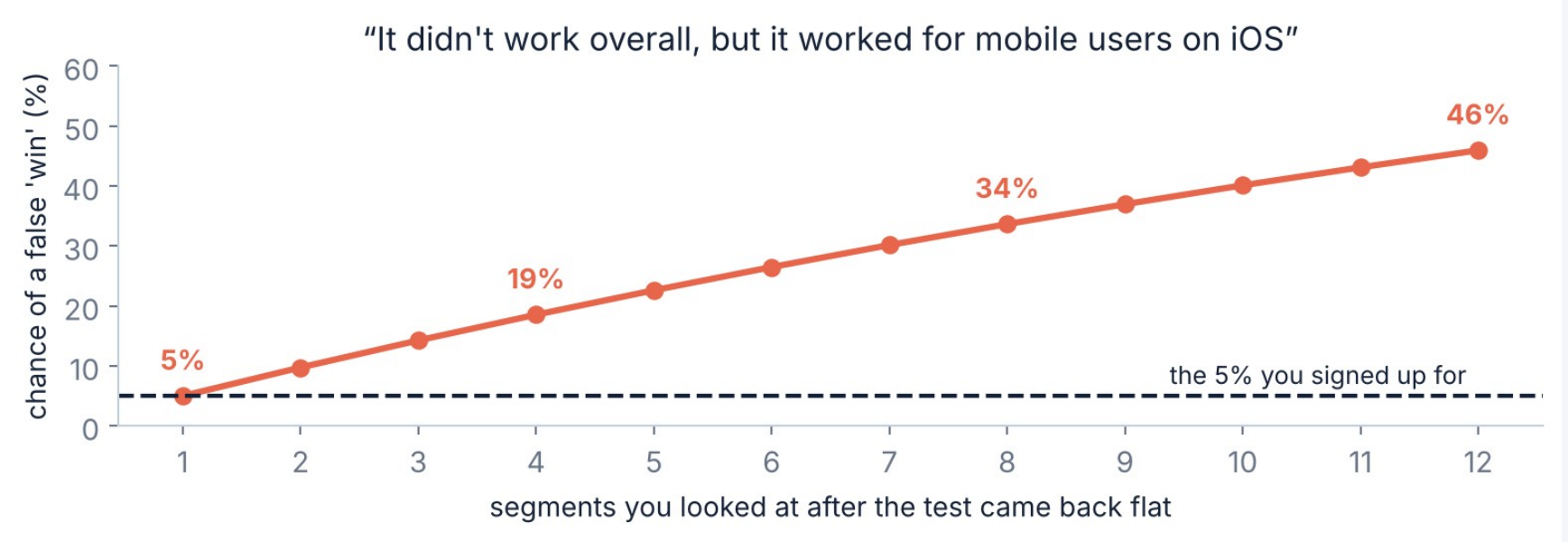
The number that should worry you

A REAL +10% revenue lift, at 1,000 users per arm: 9% power. You would miss it nine times out of ten. The same +10% on a well-behaved metric at the same n: 96% power.

What actually helps

Cap the top 1% before testing, decided in advance: power at 20,000/arm goes from 56% to 94%. Or test conversion rate instead, which is far better behaved. The false-positive rate was never the problem — it measured 3.4–4.3%, slightly under the nominal 5%.

⚠️ “It didn't work overall, but it worked for …”



Day 28 and Day 29, again

Slice a flat test eight ways and you have a 34% chance of finding a “winning” segment in data with no effect in it. Twelve ways and it is 46%.

The rule

Segments named in the plan BEFORE the test are hypotheses. Segments found afterwards are observations — report them as “worth a follow-up test”, never as a result, and correct for how many you looked at.

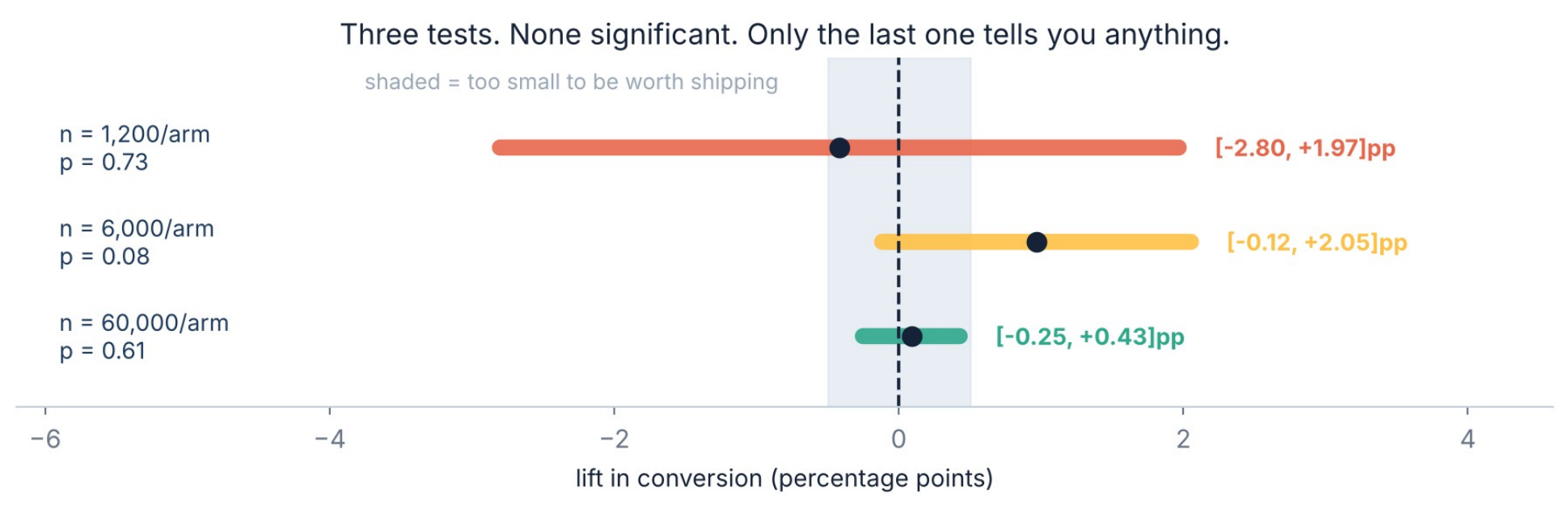
5

Deciding and Reporting

Ship, kill, or admit you learned nothing · 15 minutes

- Reading a flat result
- The decision
- The report

⭐ “Not significant” is three different results



n = 1,200: nothing

The interval allows a +2pp win and a −2.8pp disaster. You spent two weeks to learn nothing at all.

n = 6,000: still nothing

p = 0.08, and a +2pp win is still entirely compatible with the data. This is the one people wrongly call “no effect”.

n = 60,000: a real answer

Anything above +0.43pp is ruled out. Below the ship threshold, so: do not ship. That is a finding.

The decision is about the interval, not the verdict

Where the 95% interval sits	What it means	Decision
Entirely above the ship threshold	a win big enough to be worth it	SHIP
Crosses the threshold, excludes zero	real, but maybe too small to pay for	judgement call — weigh the cost
Entirely inside ± the threshold	any effect is too small to matter	KILL, and you have learned something
Wide, crossing everything	the test was too small	INCONCLUSIVE — do not call it “no effect”
Entirely below zero	the change actively hurt	KILL, and understand why

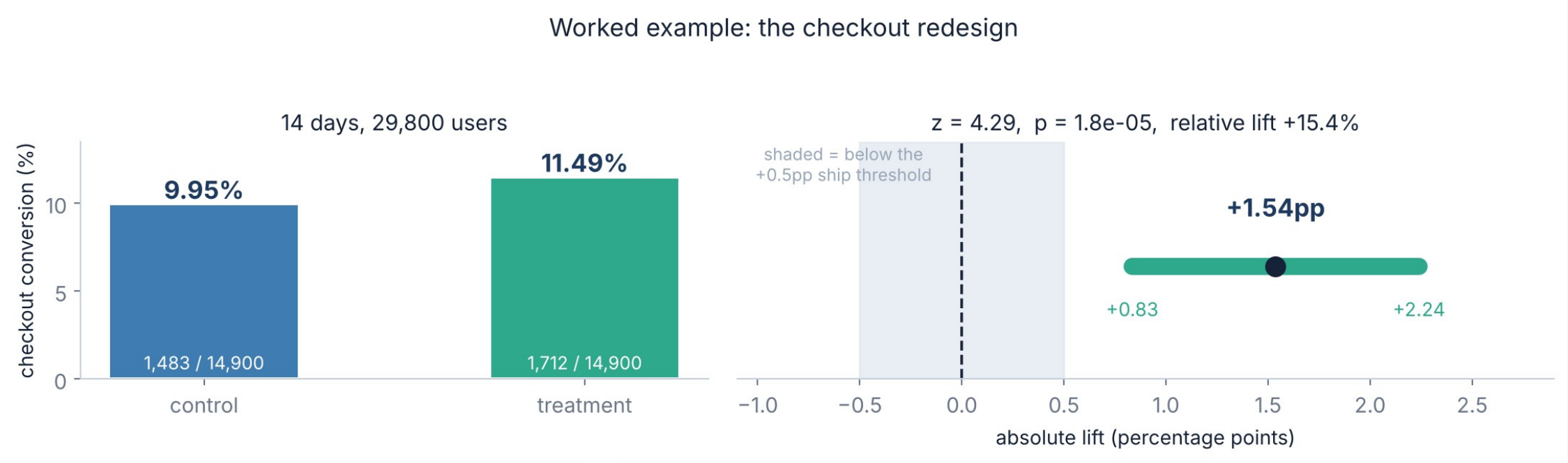
Statistical ≠ practical

A significant +0.05pp on 10 million users is real and not worth the engineering. Decide the threshold BEFORE the test, with the people who pay for it.

A kill is a success

Most tested ideas do not work. A well-powered test that rules out anything worth shipping has saved the team months, and that is the common outcome, not the failure case.

Worked through: a checkout redesign, start to finish



Planned, before any data

Baseline 10%. Smallest worthwhile lift +1pp → 14,751/arm → 7.4 days → round up to 14, two whole weeks.

Checks passed

SRM: 14,900 vs 14,900, p = 1.00. Ran 14 whole days. Nobody peeked.

Decision: SHIP

The whole interval (+0.83 to +2.24pp) sits above the +0.5pp threshold. Not just significant — big enough.

What to write when it is over

The paragraph

“We tested a checkout redesign against the current page, randomised by user, 14,900 users per arm over 14 days (planned for 14,751 to detect +1pp at 80% power). Checkout conversion rose from 9.95% to 11.49%: an absolute lift of +1.54pp, 95% CI +0.83 to +2.24pp (p = 1.8e-05), a relative lift of +15.4%. The split passed an SRM check. Revenue per user rose +2.33 (p = 0.033), which we treat as directional. The interval sits entirely above our +0.5pp ship threshold, so we recommend shipping.”

Everything it contains

- What was changed, and the unit
- n per arm, and the planned n
- Both rates, not just the lift
- Absolute lift AND its interval
- Relative lift, clearly labelled
- The exact p, not “p < 0.05”
- The SRM check
- Secondary metrics, marked as secondary
- The decision, and the threshold it was judged against

And one line that is harder to write: “We looked at N segments / ran M tests this quarter.” Nothing else in the report tells the reader how much searching went into it, and nothing else changes how they should read it as much.

RECAP

Reference — the test, the trap, the fix

Metric or situation	What to use	scipy / formula
Conversion rate, two arms	two-proportion z-test	`two_prop(ca, na, cb, nb)`
Time, items, value per user	Welch t-test	`ttest_ind(a, b, equal_var=False)`
Revenue (heavy tailed)	cap the top 1% first	`np.minimum(x, cap)`
Did the split come out right?	chi-square on the counts	`stats.chisquare([na, nb])`
How many users do I need?	power calculation	`n_per_arm(p0, mde)`
More than two arms	correct for the comparisons	`alpha / n_comparisons`
I want to stop early	sequential test, not a t-test	`# plan it in advance`

Always report together: both rates, n per arm, the absolute lift with its 95% interval, the exact p, the relative lift, and the SRM check. Six numbers, and a reader can judge the whole test.

RECAP

Debug corner — eight ways an A/B test goes wrong

You did	What goes wrong	Fix
ran it until it looked significant	20 looks → 25% false positives	fix the end date from the sum
skipped the sample-size sum	winners overstate the lift 3.3×	n_per_arm(p0, mde) — one line
randomised by pageview	one user sees both arms; p is wrong	randomise by user id
never checked the split	a 51.5/48.5 split invalidates it	chi-square on the two counts
ran Monday to Friday	the baseline swings 2.1pp at weekends	run whole weeks
stopped on day 3	novelty, not effect: +1.33pp vs +0.02	plot the daily lift, not cumulative
sliced it 8 ways after it went flat	34% chance of a fake winner	pre-specify segments, or correct
reported “no significant difference”	hides whether you learned anything	report the interval

Rows 1, 5 and 6 all disappear if you fix row 2: do the sample-size sum first, and write the end date in the calendar before the test launches.

Putting it together — one function for the whole test

```
def ab_test(c_a, n_a, c_b, n_b, ship_threshold=0.005, alpha=.05):
    """Analyse a conversion A/B test and recommend a decision."""

    #  0. SRM first - if the split is broken, nothing else matters
    tot = n_a + n_b
    srm_p = stats.chisquare([n_a, n_b], [tot/2, tot/2]).pvalue
    if srm_p < 0.01:
        return print(f"STOP: sample ratio mismatch (p={srm_p:.1e}). "
                     f"Fix the bug and rerun - do not analyse this.")

    p_a, p_b = c_a/n_a, c_b/n_b
    diff = p_b - p_a
    pbar = (c_a + c_b) / tot                          # pooled, for the test
    z = diff / np.sqrt(pbar*(1-pbar)*(1/n_a + 1/n_b))
    p = 2*(1 - stats.norm.cdf(abs(z)))
    se = np.sqrt(p_a*(1-p_a)/n_a + p_b*(1-p_b)/n_b)   # unpooled, for the CI
    lo, hi = diff - 1.96*se, diff + 1.96*se

    print(f"control {p_a:.3%} ({c_a:,}/{n_a:,}) | treat {p_b:.3%} ({c_b:,}/{n_b:,})")
    print(f"lift {diff*100:+.3f}pp  95% CI [{lo*100:+.3f}, {hi*100:+.3f}]pp")
    print(f"relative {diff/p_a:+.2%}   p = {p:.3e}   SRM p = {srm_p:.2f}")

    t = ship_threshold
    verdict = ("SHIP"          if lo >  t else
               "KILL - hurts"  if hi <  0 else
               "KILL - too small to matter" if (lo > -t and hi < t) else
               "JUDGEMENT CALL - real but maybe too small" if lo > 0 else
               "INCONCLUSIVE - underpowered, not 'no effect'")
    print("->", verdict)
```

SRM is checked first
a broken split voids everything after

Pooled for z, unpooled for CI
the two answer different questions

Both rates printed
a lift with no baseline is unreadable

The interval, not just p
it is what the decision uses

A threshold, not zero
significant ≠ worth shipping

“Inconclusive” is a verdict
and never “no effect”

What it cannot do is size the test for you before it runs. That is n_per_arm(), and it is the half that matters most.

OVER TO YOU

Practice session — 30 minutes

A **Predict the output** — 8 questions

Sizing, lift, intervals, SRM

B **Spot & fix the bug** — 4 questions

Peeking, pageview units, segment mining

C **Write the code** — 5 tasks

Size it, test it, check it, decide it

D **Challenge** — 2 tasks

Design one from scratch, then audit a bad one

Your notebook

`Day30_Practice_Questions.ipynb`

- Size it before you run it
- Check the split first
- Decide from the interval
- Solutions shared after class

Part D gives you a real-looking experiment write-up with four separate things wrong with it, and asks you to find them and say what each one does to the conclusion. That is the job.

TODAY IN TWELVE LINES · AND THE END OF MODULE 2

Recap

1. ⭐ A coin flip removes confounding, reverse causation and selection at once.
2. It does NOT remove chance — that is what the p-value and interval are for.
3. Decide three things first: the unit, ONE primary metric, and the size.
4. Randomise by USER. By pageview, one person sees both arms.
5. Halve the effect you want to catch → roughly 4× the users. +1pp = 14,751/arm.
6. ⚠️ At 11% power, winning tests overstated a +1.00pp lift as +3.32pp.
7. ⚠️ Peeking 20 times turns a 5% false-positive rate into 25%.
8. ⚠️ A 51.5/48.5 split on 40,000 users is p = 2e-9. It is a bug, not luck.
9. ⚠️ Novelty: day 1 read +1.79pp; by day 14 the daily lift was +0.02pp.
10. ⚠️ A real +10% revenue lift had 9% power at n = 1,000. Cap the top 1%.
11. ⚠️ Eight segments on a flat test → 34% chance of a fake winner.
12. Decide from the INTERVAL against a threshold. “Not significant” ≠ “no effect”.

Homework

Write a one-page test plan for a change you would actually make — unit, primary metric, MDE, n per arm, end date · Find a published A/B result and work out what is missing from the report · Take a feature you believe works and name the smallest lift that would justify it

Module 2 ends here — next: Topic 2.20

The statistics mini project: hypothesis testing on business data, then the Module 2 assessment. After that, Module 3 — Machine Learning, where today's “decide the threshold before you look” becomes train/test discipline.